```sql
-- Create the Bank database
CREATE DATABASE Bank;

-- Use the Bank database
USE Bank;

-- Create the Branch table
CREATE TABLE Branch (
    branch_id INT PRIMARY KEY,
    branch_name VARCHAR(50),
    branch_city VARCHAR(50),
    assets DECIMAL(10, 2)
);

-- Create the Customer table
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(50),
    customer_city VARCHAR(50),
    customer_street VARCHAR(100)
);

-- Create the Account table
```

```sql
CREATE TABLE Account (
    account_number INT PRIMARY KEY,
    branch_id INT,
    balance DECIMAL(10, 2),
    FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
);

-- Create the Loan table
CREATE TABLE Loan (
    loan_number INT PRIMARY KEY,
    branch_id INT,
    amount DECIMAL(10, 2),
    FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
);

-- Create the Depositor table
CREATE TABLE Depositor (
    customer_id INT,
    account_number INT,
    PRIMARY KEY (customer_id, account_number),
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (account_number) REFERENCES Account(account_number)
);
```

```sql
-- Create the Borrower table
CREATE TABLE Borrower (
    customer_id INT,
    loan_number INT,
    PRIMARY KEY (customer_id, loan_number),
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (loan_number) REFERENCES Loan(loan_number)
);
-- Insert data into the Branch table
INSERT INTO Branch VALUES (1, 'Downtown', 'New York', 1000000.00);
INSERT INTO Branch VALUES (2, 'Perryridge', 'Brooklyn', 800000.00);

-- Insert data into the Customer table
INSERT INTO Customer VALUES (101, 'John Doe', 'New York', '123 Main St');
INSERT INTO Customer VALUES (102, 'Jane Smith', 'Brooklyn', '456 Oak St');

-- Insert data into the Account table
INSERT INTO Account VALUES (1001, 1, 5000.00);
INSERT INTO Account VALUES (1002, 2, 3000.00);

-- Insert data into the Loan table
INSERT INTO Loan VALUES (2001, 1, 1500.00);
```

INSERT INTO Loan VALUES (2002, 2, 2000.00);

-- Insert data into the Depositor table

INSERT INTO Depositor VALUES (101, 1001);

INSERT INTO Depositor VALUES (102, 1002);

-- Insert data into the Borrower table

INSERT INTO Borrower VALUES (101, 2001);

INSERT INTO Borrower VALUES (102, 2002);

Here are the SQL queries for the tasks you requested:

1. Create a new table:

SQL
```
CREATE TABLE table_name (
  column1 data_type,
  column2 data_type,
  ...
);
```

2. Insert a new record into a table:

SQL
```
INSERT INTO table_name (column1, column2, ...)
VALUES (value1, value2, ...);
```

3. Delete records from a table:

SQL
```
DELETE FROM table_name
WHERE condition;
```

4. Update records in a table:

SQL
```
UPDATE table_name
SET column1 = new_value, column2 = new_value, ...
WHERE condition;
```

5. Add a new column to a table:

SQL
```
ALTER TABLE table_name
ADD COLUMN new_column data_type;
```

6. Delete a column from a table:

SQL
```
ALTER TABLE table_name
DROP COLUMN column_name;
```

7. Rename a column of a table:

SQL
```
ALTER TABLE table_name
RENAME COLUMN old_column_name TO new_column_name;
```

8. Select all records from a table:

SQL
```
SELECT * FROM table_name;
```

9. Select specific columns from a table:

SQL
```
SELECT column1, column2, ... FROM table_name;
```

10. Select distinct values from a column:

SQL
```
SELECT DISTINCT column_name FROM table_name;
```

11. Filter records using a WHERE clause:

SQL
```
SELECT * FROM table_name
WHERE condition;
```

12. Sort records in descending order:

SQL
```
SELECT * FROM table_name
ORDER BY column_name DESC;
```

13. Sort records in ascending order:

SQL
```
SELECT * FROM table_name
ORDER BY column_name ASC;
```

14. Join two tables based on a common column:

SQL
```
SELECT * FROM table1
INNER JOIN table2 ON table1.column_name = table2.column_name;
```

15. Count the number of rows in a table:

SQL
```
SELECT COUNT(*) FROM table_name;
```

16. Group records and calculate aggregate functions:

SQL
```
SELECT column_name, AVG(value_column), COUNT(*)
FROM table_name
GROUP BY column_name;
```

17. Limit the number of rows returned in a result set:

SQL
```
SELECT * FROM table_name
LIMIT number_of_rows;
```

18. Find the sum of values in a column:

SQL
```
SELECT SUM(column_name) FROM table_name;
```

19. Find the average value of a column:

SQL
```
SELECT AVG(column_name) FROM table_name;
```

20. Get the minimum value from a column:

SQL
```
SELECT MIN(column_name) FROM table_name;
```

21. Retrieve the maximum value from a column:

SQL
```
SELECT MAX(column_name) FROM table_name;
```

22. Retrieve rows with values within a specified range:

SQL
```
SELECT * FROM table_name
WHERE column_name BETWEEN lower_bound AND upper_bound;
```

23. Retrieve rows with values matching a list of specified values:

SQL
```
SELECT * FROM table_name
WHERE column_name IN (value1, value2, value3);
```

24. Search for patterns in a column using wildcard characters:

SQL
```
SELECT * FROM table_name
WHERE column_name LIKE 'abc%';
```

25. Filter data based on aggregate functions in a GROUP BY query:

SQL
```
SELECT column_name, COUNT(*)
FROM table_name
GROUP BY column_name
HAVING COUNT(*) > 10;
```

26. Delete all rows from a table, but keep the table structure:

SQL
```
DELETE FROM table_name;
```

27. Implement simple and complex views:

SQL
```
CREATE VIEW view_name AS
SELECT * FROM table_name
WHERE condition;
```

28. Implement stored procedures:

SQL
```
CREATE PROCEDURE procedure_name (parameters)
BEGIN
  -- SQL statements to execute
END;
```

29. Implement triggers:

SQL
```
CREATE TRIGGER trigger_name
```

```
AFTER INSERT/UPDATE/DELETE ON table_name
FOR EACH ROW
BEGIN
```

1. Consider the following relational database :

Orders(order_no, purchase_amount, order_date, customer_id, salesman_id)

Customers(customers_id, customer_name, city, grade, salesman_id)

Salesman(salesman_id, name, city, commission)

a) Write a sql statement to make a list with order no, purchase amount, customer name and their cities for those orders which order amount between 500 and 2000.

b) Write a sql statement to display those orders which are not issued on date 2024-01-05 and issued by the salesman whose id is 505.

c) Write a query to find the sums of amount from the orders table grouped by date, eliminating all those dates where the sum was not atleast 1000.

d) Write a query to display all those orders by the customers not located in the same cities where their salesman live.

```
CREATE DATABASE sales_management;

USE sales_management;


-- Create the tables

CREATE TABLE Orders (
```

```
    order_no INT PRIMARY KEY,
    purchase_amount DECIMAL(10, 2),
    order_date DATE,
    customer_id INT,
    salesman_id INT
);

CREATE TABLE Customers (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(255),
    city VARCHAR(255),
    grade VARCHAR(10),
    salesman_id INT
);

CREATE TABLE Salesman (
    salesman_id INT PRIMARY KEY,
    name VARCHAR(255),
    city VARCHAR(255),
    commission DECIMAL(5, 2)
);

-- Insert dummy values
INSERT INTO Orders VALUES
(1, 1500.00, '2024-01-01', 101, 501),
(2, 800.00, '2024-01-02', 102, 502),
(3, 2500.00, '2024-01-03', 103, 503),
(4, 1200.00, '2024-01-04', 104, 504),
(5, 1800.00, '2024-01-05', 105, 505);
```

```
INSERT INTO Customers VALUES

(101, 'Customer A', 'City1', 'A', 501),

(102, 'Customer B', 'City2', 'B', 502),

(103, 'Customer C', 'City3', 'C', 503),

(104, 'Customer D', 'City4', 'D', 504),

(105, 'Customer E', 'City5', 'E', 505);


INSERT INTO Salesman VALUES

(501, 'Salesman X', 'City1', 0.10),

(502, 'Salesman Y', 'City2', 0.12),

(503, 'Salesman Z', 'City3', 0.15),

(504, 'Salesman P', 'City4', 0.08),

(505, 'Salesman Q', 'City5', 0.18);
```

a) Orders between 500 and 2000:

SQL

```
SELECT o.order_no, o.purchase_amount, c.customer_name, c.city
FROM Orders o
INNER JOIN Customers c ON o.customer_id = c.customers_id
WHERE o.purchase_amount BETWEEN 500 AND 2000;
```

b) Orders not issued on 2024-01-05 by salesman 505:

SQL

```
SELECT *
FROM Orders o
WHERE o.order_date != '2024-01-05' AND o.salesman_id = 505;
```

c) Sums of order amounts by date with a minimum sum of 1000:

SQL

```
SELECT order_date, SUM(purchase_amount) AS total_amount
FROM Orders
GROUP BY order_date
HAVING SUM(purchase_amount) >= 1000;
```

d) Orders by customers in different cities from their salesman:

SQL

```
SELECT o.order_no, c.customer_name, c.city, s.city AS salesman_city
FROM Orders o
```

```
INNER JOIN Customers c ON o.customer_id = c.customers_id
INNER JOIN Salesman s ON o.salesman_id = s.salesman_id
WHERE c.city <> s.city;
```

2. Consider the following database schema :

Employee(emp_id, name, hire_date, job_id, salary, commission, manager_id,

department_id)

Departments(department_id, department_name, manager_id, location_id)

Locations(location_id, street, post_code, city, state, country)

a) Write a query to display the department id, employee name, job and department name for all employees in the Finanace Department.

b) Write a query to display all the information of the employees who do not work on those departments where some employees works whose id within the range 100 and 200.

c) Write a query to display the employee id, name and job for all employees whose salary is more than any average salary of any department.

d) Write a query to display the department id and the total salary for those

departments which contain at least one salaried employee.

```
CREATE DATABASE company_management;

USE company_management;


-- Create the tables

CREATE TABLE Employee (

    emp_id INT PRIMARY KEY,

    name VARCHAR(255),

    hire_date DATE,

    job_id VARCHAR(10),

    salary DECIMAL(10, 2),

    commission DECIMAL(5, 2),

    manager_id INT,

    department_id INT

);
```

```sql
CREATE TABLE Departments (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(255),
    manager_id INT,
    location_id INT
);

CREATE TABLE Locations (
    location_id INT PRIMARY KEY,
    street VARCHAR(255),
    post_code VARCHAR(20),
    city VARCHAR(255),
    state VARCHAR(255),
    country VARCHAR(255)
);

-- Insert dummy values
INSERT INTO Employee VALUES
(101, 'John Doe', '2022-01-01', 'Manager', 60000.00, NULL, NULL, 1),
(102, 'Jane Smith', '2022-02-01', 'Analyst', 50000.00, NULL, 101, 2),
(103, 'Bob Johnson', '2022-03-01', 'Clerk', 40000.00, NULL, 101, 1),
(104, 'Alice Brown', '2022-04-01', 'Analyst', 55000.00, NULL, 101, 2),
(105, 'Charlie Davis', '2022-05-01', 'Manager', 70000.00, NULL, NULL, 3);

INSERT INTO Departments VALUES
(1, 'Finance', 101, 1),
(2, 'HR', 105, 2),
(3, 'IT', NULL, 3);

INSERT INTO Locations VALUES
```

```
(1, 'Street1', '12345', 'City1', 'State1', 'Country1'),

(2, 'Street2', '67890', 'City2', 'State2', 'Country2'),

(3, 'Street3', '54321', 'City3', 'State3', 'Country3');
```

a) Employees in the Finance Department:

SQL

```
SELECT e.emp_id, e.name, j.job_title, d.department_name
FROM Employee e
INNER JOIN Departments d ON e.department_id = d.department_id
INNER JOIN Jobs j ON e.job_id = j.job_id
WHERE d.department_name = 'Finance';
```

b) Employees not working in departments with ID 100 to 200:

SQL

```
SELECT e.emp_id, e.name, j.job_title
FROM Employee e
INNER JOIN Jobs j ON e.job_id = j.job_id
INNER JOIN Departments d ON e.department_id = d.department_id
WHERE d.department_id NOT IN (SELECT department_id FROM Employee WHERE emp_id
BETWEEN 100 AND 200);
```

c) Employees with salary higher than any department average:

SQL

```
SELECT e.emp_id, e.name, j.job_title
FROM Employee e
INNER JOIN Jobs j ON e.job_id = j.job_id
WHERE e.salary > (
  SELECT AVG(salary)
  FROM Employee
  GROUP BY department_id
  ORDER BY AVG(salary) DESC
  LIMIT 1
);
```

d) Departments with at least one salaried employee:

SQL

```
SELECT d.department_id, SUM(e.salary) AS total_salary
FROM Departments d
INNER JOIN Employee e ON d.department_id = e.department_id
WHERE e.salary IS NOT NULL
GROUP BY d.department_id;
```

3. Consider the following relational schema :

```
Employee(emp_id, street, city)

Works(emp_id, company_name, salary)
```

Company(company_id, company_name, city)

a) Find the name of all employees who live in the same city as the company for which

they work)

b) Find the name of all employees who do not work for the AB Bank.

c) Find the name of all employees who earn more than every employee of SB

Limited.

d) Find the name of all employees who work for FB limited.

```
CREATE DATABASE employment_management;

USE employment_management;


-- Create the tables

CREATE TABLE Employee (

    emp_id INT PRIMARY KEY,

    street VARCHAR(255),

    city VARCHAR(255)

);


CREATE TABLE Works (

    emp_id INT,

    company_name VARCHAR(255),

    salary DECIMAL(10, 2),

    PRIMARY KEY (emp_id, company_name),

    FOREIGN KEY (emp_id) REFERENCES Employee(emp_id)

);


CREATE TABLE Company (

    company_id INT PRIMARY KEY,

    company_name VARCHAR(255),

    city VARCHAR(255)

);
```

```
-- Insert dummy values

INSERT INTO Employee VALUES

(101, 'Street1', 'City1'),

(102, 'Street2', 'City2'),

(103, 'Street3', 'City1'),

(104, 'Street4', 'City3'),

(105, 'Street5', 'City2');


INSERT INTO Company VALUES

(1, 'AB Bank', 'City1'),

(2, 'SB Limited', 'City2'),

(3, 'FB Limited', 'City3');


INSERT INTO Works VALUES

(101, 'AB Bank', 60000.00),

(102, 'SB Limited', 50000.00),

(103, 'FB Limited', 70000.00),

(104, 'AB Bank', 55000.00),

(105, 'SB Limited', 48000.00);
```

a) Employees in the same city as their company:

SQL

```
SELECT e.street, e.city, w.company_name
FROM Employee e
INNER JOIN Works w ON e.emp_id = w.emp_id
INNER JOIN Company c ON w.company_name = c.company_name
WHERE e.city = c.city;
```

b) Employees not working for AB Bank:

SQL

```
SELECT e.street, e.city
FROM Employee e
LEFT JOIN Works w ON e.emp_id = w.emp_id
WHERE w.company_name IS NULL OR w.company_name <> 'AB Bank';
```

c) Employees earning more than all SB Limited employees:

SQL

```
SELECT e.street, e.city
FROM Employee e
WHERE e.emp_id NOT IN (
  SELECT w.emp_id
  FROM Works w
  WHERE w.company_name = 'SB Limited'
) OR e.emp_id IN (
  SELECT w.emp_id
  FROM Works w
  WHERE w.company_name = 'SB Limited'
  HAVING e.salary > MAX(w.salary)
);
```

d) Employees working for FB Limited:

SQL

```
SELECT e.street, e.city
FROM Employee e
INNER JOIN Works w ON e.emp_id = w.emp_id
WHERE w.company_name = 'FB Limited';
```

4. Consider the following relational schema :

Student(student_id, name, fathers_name, admission_date, reg_no)

Courses(course_id, course_title, course_type, credit, dept_id)

Major(student_id, department_id, admission_session)

CourseRegistration(student_id, course_id, year, semester, session)

CourseComplete(student_id, course_id, year, semester, session, grade_point, grade_letter)

a) Display the students of “CSE” department of 1st Year and 2nd Semester.

b) List the students of EEE who have completed 5 course.

c) List the students who get grade point below 2.50 in any courses in session 2017- 2018

d) List the student id, name and fathers name of students who go A+ in course “CSE 2201”.

CREATE DATABASE student_management;

USE student_management;

```sql
-- Create the tables
CREATE TABLE Student (
    student_id INT PRIMARY KEY,
    name VARCHAR(255),
    fathers_name VARCHAR(255),
    admission_date DATE,
    reg_no VARCHAR(20)
);

CREATE TABLE Courses (
    course_id INT PRIMARY KEY,
    course_title VARCHAR(255),
    course_type VARCHAR(50),
    credit INT,
    dept_id INT
);

CREATE TABLE Major (
    student_id INT,
    department_id INT,
    admission_session VARCHAR(20),
    PRIMARY KEY (student_id, department_id),
    FOREIGN KEY (student_id) REFERENCES Student(student_id),
    FOREIGN KEY (department_id) REFERENCES Courses(dept_id)
);
```

```
CREATE TABLE CourseRegistration (
    student_id INT,
    course_id INT,
    year INT,
    semester INT,
    session VARCHAR(20),
    PRIMARY KEY (student_id, course_id, year, semester),
    FOREIGN KEY (student_id) REFERENCES Student(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);

CREATE TABLE CourseComplete (
    student_id INT,
    course_id INT,
    year INT,
    semester INT,
    session VARCHAR(20),
    grade_point DECIMAL(4, 2),
    grade_letter VARCHAR(2),
    PRIMARY KEY (student_id, course_id, year, semester),
    FOREIGN KEY (student_id) REFERENCES Student(student_id),
    FOREIGN KEY (course_id) REFERENCES Courses(course_id)
);
```

```
-- Insert dummy values

INSERT INTO Student VALUES

(101, 'John Doe', 'John Doe Sr.', '2022-01-01', '2022001'),

(102, 'Jane Smith', 'John Smith', '2022-01-15', '2022002'),

(103, 'Bob Johnson', 'Mike Johnson', '2022-02-01', '2022003'),

(104, 'Alice Brown', 'David Brown', '2022-02-15', '2022004'),

(105, 'Charlie Davis', 'Michael Davis', '2022-03-01', '2022005');


INSERT INTO Courses VALUES

(201, 'CSE 1101', 'Theory', 3, 1),

(202, 'CSE 2201', 'Theory', 3, 1),

(203, 'EEE 1101', 'Theory', 3, 2),

(204, 'EEE 2201', 'Theory', 3, 2),

(205, 'MAT 1101', 'Theory', 3, 3);


INSERT INTO Major VALUES

(101, 1, '2022'),

(102, 1, '2022'),

(103, 2, '2022'),

(104, 2, '2022'),

(105, 3, '2022');


INSERT INTO CourseRegistration VALUES

(101, 201, 2022, 1, 'Spring'),

(101, 202, 2022, 2, 'Fall'),
```

(102, 201, 2022, 1, 'Spring'),

(102, 202, 2022, 2, 'Fall'),

(103, 203, 2022, 1, 'Spring'),

(103, 204, 2022, 2, 'Fall'),

(104, 203, 2022, 1, 'Spring'),

(104, 204, 2022, 2, 'Fall'),

(105, 205, 2022, 1, 'Spring');

INSERT INTO CourseComplete VALUES

(101, 201, 2022, 1, 'Spring', 3.75, 'A'),

(101, 202, 2022, 2, 'Fall', 3.50, 'A-'),

(102, 201, 2022, 1, 'Spring', 3.80, 'A'),

(102, 202, 2022, 2, 'Fall', 3.70, 'A-'),

(103, 203, 2022, 1, 'Spring', 3.25, 'B+'),

(103, 204, 2022, 2, 'Fall', 3.00, 'B'),

(104, 203, 2022, 1, 'Spring', 3.40, 'A-'),

(104, 204, 2022, 2, 'Fall', 3.60, 'A'),

(105, 205, 2022, 1, 'Spring', 3.90, 'A');

a) Students in CSE department, 1st Year, 2nd Semester:

SQL

```
SELECT s.student_id, s.name, s.fathers_name
FROM Student s
INNER JOIN Major m ON s.student_id = m.student_id
INNER JOIN CourseRegistration cr ON s.student_id = cr.student_id
WHERE m.department_id = 'CSE' AND cr.year = 1 AND cr.semester = 2
GROUP BY s.student_id, s.name, s.fathers_name;
```

b) Students of EEE who completed 5 courses:

SQL

```
SELECT s.student_id, s.name, s.fathers_name
```

```
FROM Student s
INNER JOIN CourseComplete cc ON s.student_id = cc.student_id
GROUP BY s.student_id, s.name, s.fathers_name
HAVING COUNT(DISTINCT cc.course_id) = 5;
```

c) Students with grade point below 2.5 in session 2017-2018:

SQL

```
SELECT DISTINCT s.student_id, s.name, s.fathers_name
FROM Student s
INNER JOIN CourseComplete cc ON s.student_id = cc.student_id
WHERE cc.session = '2017-2018' AND cc.grade_point < 2.5;
```

d) Students with A+ in CSE 2201:

SQL

```
SELECT DISTINCT s.student_id, s.name, s.fathers_name
FROM Student s
INNER JOIN CourseComplete cc ON s.student_id = cc.student_id
WHERE cc.course_id = 'CSE 2201' AND cc.grade_letter = 'A+';
```

5. Consider an insurance database that maintain the following information :

Person(DriverNo, Name, Address)

Car(LicenseNo, Model, Year)

Accident(ReportNo, Date, Location)

Owns(DriverNo, LicenseNo)

Participated(DriverNo, LicenseNo, ReportNo, DamageAmount)

Write down the SQL command to find the following queries :

i. Find the total number of people whose cars were involved in accident.

ii. Find the number of accident in which the belongs to “Mr. X”.

iii. Delete the “CAR1” car belongs to “Mr. X”.

iv. Add new accident report for the “CAR2” car belongs to “Mr. Y”.

CREATE DATABASE accident_management;

USE accident_management;

-- Create the tables

```
CREATE TABLE Person (
   DriverNo INT PRIMARY KEY,
   Name VARCHAR(255),
   Address VARCHAR(255)
);

CREATE TABLE Car (
   LicenseNo VARCHAR(20) PRIMARY KEY,
   Model VARCHAR(255),
   Year INT
);

CREATE TABLE Accident (
   ReportNo INT PRIMARY KEY,
   Date DATE,
   Location VARCHAR(255)
);

CREATE TABLE Owns (
   DriverNo INT,
   LicenseNo VARCHAR(20),
   PRIMARY KEY (DriverNo, LicenseNo),
   FOREIGN KEY (DriverNo) REFERENCES Person(DriverNo),
   FOREIGN KEY (LicenseNo) REFERENCES Car(LicenseNo)
);
```

```
CREATE TABLE Participated (
    DriverNo INT,
    LicenseNo VARCHAR(20),
    ReportNo INT,
    DamageAmount DECIMAL(10, 2),
    PRIMARY KEY (DriverNo, LicenseNo, ReportNo),
    FOREIGN KEY (DriverNo) REFERENCES Person(DriverNo),
    FOREIGN KEY (LicenseNo) REFERENCES Car(LicenseNo),
    FOREIGN KEY (ReportNo) REFERENCES Accident(ReportNo)
);

-- Insert dummy values
INSERT INTO Person VALUES
(101, 'Mr. X', 'Address1'),
(102, 'Mr. Y', 'Address2'),
(103, 'Mr. Z', 'Address3');

INSERT INTO Car VALUES
('CAR1', 'Model1', 2019),
('CAR2', 'Model2', 2020),
('CAR3', 'Model3', 2018);

INSERT INTO Accident VALUES
(1001, '2022-01-01', 'Location1'),
```

(1002, '2022-02-01', 'Location2'),

(1003, '2022-03-01', 'Location3');

INSERT INTO Owns VALUES

(101, 'CAR1'),

(102, 'CAR2'),

(103, 'CAR3');

INSERT INTO Participated VALUES

(101, 'CAR1', 1001, 5000.00),

(102, 'CAR2', 1002, 7000.00),

(103, 'CAR3', 1003, 4500.00);

i. Total number of people with cars in accidents:

SQL

```
SELECT COUNT(DISTINCT p.DriverNo)
FROM Person p
INNER JOIN Owns o ON p.DriverNo = o.DriverNo
INNER JOIN Participated pt ON o.LicenseNo = pt.LicenseNo;
```

ii. Number of accidents involving "Mr. X"'s car:

SQL

```
SELECT COUNT(DISTINCT a.ReportNo)
FROM Accident a
INNER JOIN Participated p ON a.ReportNo = p.ReportNo
INNER JOIN Owns o ON p.LicenseNo = o.LicenseNo
INNER JOIN Person per ON o.DriverNo = per.DriverNo
WHERE per.Name = 'Mr. X';
```

iii. Delete "CAR1" belonging to "Mr. X":

SQL

```
DELETE FROM Car
WHERE LicenseNo = 'CAR1' AND EXISTS (
  SELECT 1 FROM Person p
  INNER JOIN Owns o ON p.DriverNo = o.DriverNo
  WHERE o.LicenseNo = 'CAR1' AND p.Name = 'Mr. X'
);
```

iv. Add new accident report for "CAR2" belonging to "Mr. Y":

SQL

```
INSERT INTO Accident (ReportNo, Date, Location)
VALUES ('NewReport', CURRENT_DATE, 'Accident Location');

INSERT INTO Participated (DriverNo, LicenseNo, ReportNo, DamageAmount)
SELECT p.DriverNo, 'CAR2', 'NewReport', 0
FROM Person p
INNER JOIN Owns o ON p.DriverNo = o.DriverNo
WHERE o.LicenseNo = 'CAR2' AND p.Name = 'Mr. Y';
```

6. Consider the following relational schema :

Branch(branch_id, branch_name, branch_city, assets)

Customer(customer_id, customer_name, customer_city, customer_street)

Account(account_number, branch_id, balance)

Loan(loan_number, branch_id, amount)

Depositor(customer_id, account_number)

Borrower(customer_id, loan_number)

a) Find the loan number for each loan of an amount greater than $1200.

b) Find the names of all customers who have a loan, an account, or both from the bank.

c) Find the name of all customers who have a loan at the perryridge branch.

d) Find the name of all customers who an account at every branch located in

Brroklyn.

```
CREATE DATABASE bank_management;

USE bank_management;


-- Create the tables

CREATE TABLE Branch (

    branch_id INT PRIMARY KEY,

    branch_name VARCHAR(255),

    branch_city VARCHAR(255),

    assets DECIMAL(15, 2)

);
```

```sql
CREATE TABLE Customer (
    customer_id INT PRIMARY KEY,
    customer_name VARCHAR(255),
    customer_city VARCHAR(255),
    customer_street VARCHAR(255)
);

CREATE TABLE Account (
    account_number INT PRIMARY KEY,
    branch_id INT,
    balance DECIMAL(15, 2),
    FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
);

CREATE TABLE Loan (
    loan_number INT PRIMARY KEY,
    branch_id INT,
    amount DECIMAL(15, 2),
    FOREIGN KEY (branch_id) REFERENCES Branch(branch_id)
);

CREATE TABLE Depositor (
    customer_id INT,
    account_number INT,
    PRIMARY KEY (customer_id, account_number),
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (account_number) REFERENCES Account(account_number)
);
```

```sql
CREATE TABLE Borrower (
    customer_id INT,
    loan_number INT,
    PRIMARY KEY (customer_id, loan_number),
    FOREIGN KEY (customer_id) REFERENCES Customer(customer_id),
    FOREIGN KEY (loan_number) REFERENCES Loan(loan_number)
);

-- Insert dummy values
INSERT INTO Branch VALUES
(1, 'Perryridge', 'Brooklyn', 150000.00),
(2, 'Downtown', 'Manhattan', 200000.00),
(3, 'Uptown', 'Bronx', 180000.00);

INSERT INTO Customer VALUES
(101, 'John Doe', 'Brooklyn', 'Street1'),
(102, 'Jane Smith', 'Manhattan', 'Street2'),
(103, 'Bob Johnson', 'Brooklyn', 'Street3'),
(104, 'Alice Brown', 'Bronx', 'Street4');

INSERT INTO Account VALUES
(1001, 1, 5000.00),
(1002, 2, 7000.00),
(1003, 1, 3000.00),
(1004, 3, 9000.00);

INSERT INTO Loan VALUES
(2001, 1, 1500.00),
(2002, 2, 2500.00),
(2003, 1, 1800.00),
```

```
(2004, 3, 1200.00);


INSERT INTO Depositor VALUES

(101, 1001),

(102, 1002),

(103, 1003),

(104, 1004);


INSERT INTO Borrower VALUES

(101, 2001),

(102, 2002),

(103, 2003),

(104, 2004);
```

a) Loan numbers for loans exceeding $1200:

SQL

```
SELECT loan_number
FROM Loan
WHERE amount > 1200;
```

b) Customers with a loan, account, or both:

SQL

```
SELECT DISTINCT customer_name
FROM Customer c
LEFT JOIN Borrower b ON c.customer_id = b.customer_id
LEFT JOIN Depositor d ON c.customer_id = d.customer_id
WHERE b.customer_id IS NOT NULL OR d.customer_id IS NOT NULL;
```

c) Customers with a loan at Perryridge branch:

SQL

```
SELECT c.customer_name
FROM Customer c
INNER JOIN Borrower b ON c.customer_id = b.customer_id
INNER JOIN Loan l ON b.loan_number = l.loan_number
INNER JOIN Branch br ON l.branch_id = br.branch_id
WHERE br.branch_city = 'Perryridge';
```

d) Customers with an account at every Brooklyn branch:

SQL

```
SELECT DISTINCT c.customer_name
FROM Customer c
INNER JOIN Depositor d ON c.customer_id = d.customer_id
INNER JOIN Account a ON d.account_number = a.account_number
INNER JOIN Branch br ON a.branch_id = br.branch_id
WHERE br.branch_city = 'Brooklyn'
GROUP BY c.customer_id
HAVING COUNT(DISTINCT br.branch_id) = (
  SELECT COUNT(DISTINCT branch_id)
  FROM Branch
  WHERE branch_city = 'Brooklyn'
);
```

employees(emp_id, name, department, salary, email)

projects(project_id, project_name, location_id)

emp_projects(emp_id, project_id)

locations(location id, location_name)

1. Create a database name "employee_management".

2. Create above four tables with appropriate primary key and foreign key.

3. Insert at least five values to all tables given above.

4. Write a query to display all the project name and project location name.

5. Write a query to find employee name who is taking second highest salary using sub query concept.

6. Write a SQL statement to change the email column of employees table with 'not available for those employees who works for "HR" department.

7. Write a SQL statement to add a column "project_type" to projects table.

8. Write a query using trigger concept to store (emp_id, name and department) to a new table named emp_log on each and every insertion happened on employees table.

9. Write a query using trigger concept to store (project_id and project_name) to a new table named project_log on each and every deletion happened on projects table.

10)Write a query to illustrate the concept of parameterized stored procedure.

11)Write a query to illustrate the concept of complex views.

12)Write queries to illustrate the concept of access control

1)

```
CREATE DATABASE employee_management;

USE employee_management;
```

2)

```
CREATE TABLE employees (

    emp_id INT PRIMARY KEY,

    name VARCHAR(255),

    department VARCHAR(255),

    salary DECIMAL(10, 2),

    email VARCHAR(255)

);


CREATE TABLE projects (

    project_id INT PRIMARY KEY,

    project_name VARCHAR(255),

    location_id INT

);


CREATE TABLE emp_projects (

    emp_id INT,

    project_id INT,

    PRIMARY KEY (emp_id, project_id),

    FOREIGN KEY (emp_id) REFERENCES employees(emp_id),

    FOREIGN KEY (project_id) REFERENCES projects(project_id)
```

```
);

CREATE TABLE locations (
    location_id INT PRIMARY KEY,
    location_name VARCHAR(255)
);
```

3

```
-- Insert into employees
INSERT INTO employees VALUES
(1, 'John Doe', 'IT', 60000.00, 'john.doe@example.com'),
(2, 'Jane Smith', 'HR', 50000.00, 'jane.smith@example.com'),
(3, 'Bob Johnson', 'IT', 70000.00, 'bob.johnson@example.com'),
(4, 'Alice Brown', 'Finance', 55000.00, 'alice.brown@example.com'),
(5, 'Charlie Davis', 'HR', 48000.00, 'charlie.davis@example.com');

-- Insert into projects
INSERT INTO projects VALUES
(101, 'Project A', 1),
(102, 'Project B', 2),
(103, 'Project C', 3),
(104, 'Project D', 1),
(105, 'Project E', 2);

-- Insert into emp_projects
INSERT INTO emp_projects VALUES
```

```
(1, 101),
(2, 102),
(3, 103),
(4, 104),
(5, 105);

-- Insert into locations
INSERT INTO locations VALUES
(1, 'Location X'),
(2, 'Location Y'),
(3, 'Location Z');
```

4

```
SELECT p.project_name, l.location_name
FROM projects p
JOIN locations l ON p.location_id = l.location_id;
```

5

```
SELECT name
FROM employees
WHERE salary = (
   SELECT MAX(salary)
   FROM employees
   WHERE salary < (SELECT MAX(salary) FROM employees)
);
```

6

```
UPDATE employees
```

```
SET email = 'not available'

WHERE department = 'HR';
```

7

```
ALTER TABLE projects

ADD COLUMN project_type VARCHAR(255);
```

8

```
CREATE TABLE emp_log (

   emp_id INT,

   name VARCHAR(255),

   department VARCHAR(255),

   log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,

   PRIMARY KEY (emp_id, log_time)

);


DELIMITER //

CREATE TRIGGER emp_insert_trigger AFTER INSERT ON employees

FOR EACH ROW

BEGIN

   INSERT INTO emp_log (emp_id, name, department) VALUES (NEW.emp_id, NEW.name,
NEW.department);

END;

//

DELIMITER ;
```

9

```
CREATE TABLE project_log (

   project_id INT,
```

```
    project_name VARCHAR(255),
    log_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    PRIMARY KEY (project_id, log_time)
);

DELIMITER //
CREATE TRIGGER project_delete_trigger AFTER DELETE ON projects
FOR EACH ROW
BEGIN
    INSERT INTO project_log (project_id, project_name) VALUES (OLD.project_id, OLD.project_name);
END;
//
DELIMITER ;
```

10

```
-- Create a parameterized stored procedure
DELIMITER //
CREATE PROCEDURE GetEmployeeByDepartment(IN department_name VARCHAR(255))
BEGIN
    SELECT * FROM employees WHERE department = department_name;
END;
//
DELIMITER ;

-- Call the stored procedure with a parameter
CALL GetEmployeeByDepartment('IT');
```

11

```
-- Create a complex view

CREATE VIEW EmployeeProjectView AS

SELECT e.emp_id, e.name, e.department, p.project_name

FROM employees e

JOIN emp_projects ep ON e.emp_id = ep.emp_id

JOIN projects p ON ep.project_id = p.project_id;
```

12

```
-- Grant SELECT permission to a user on the employees table

GRANT SELECT ON employee_management.employees TO 'username'@'localhost';


-- Revoke INSERT permission from a user on the projects table

REVOKE INSERT ON employee_management.projects FROM 'username'@'localhost';
```

Customer (Customer-ID, Name, Address, Age)

Order (Customer_ID, Product_ID, Date)

Product (Product-ID, Name, Size, Price)

1 Find all Customers' Names who live in Dhaka/do not live in Chittagong.

ii. Find the Customer_ID and Name of all Customers whose Age is over 20/greater than or equal to 22/less than 25/equal to 30.

Add 1000 TK Price for all of the Products in this database.

iv Add 10% VAT for all of the Products in this database.

V. Find all product Name starting with 'A'/ ending with "/ 'A' at the second position/"A' in any position/start with 'S' and end with "T.

vi . Find Customers' Name, Age, and Order Date from this database.

vii. Find all Customers in the database whose Name is 'Faria' and Age 17.

viii. Find the Product ID and Size of all Products where the Price is between 200 and 1000.

ix. Modify the Customer Table in the database so that 'Jony' now lives in 'Sylhet.

X. Delete All information about 'Safin' from the Customers' Table.

1

```
SELECT Name

FROM Customer

WHERE Address = 'Dhaka' OR Address != 'Chittagong';
```

2

```
SELECT Customer_ID, Name

FROM Customer

WHERE Age > 20 OR Age >= 22 OR Age < 25 OR Age = 30;
```

3

```
UPDATE Product

SET Price = Price + 1000;
```

4

```
UPDATE Product

SET Price = Price * 1.1;
```

5

```
SELECT Name

FROM Product

WHERE Name LIKE 'A%' OR Name LIKE '%/' OR Name LIKE '_A%' OR Name LIKE '%A%' OR Name LIKE 'S%T';
```

6

```
SELECT C.Name, C.Age, O.Date

FROM Customer C

JOIN Order O ON C.Customer_ID = O.Customer_ID;
```

7

```
SELECT *

FROM Customer

WHERE Name = 'Faria' AND Age = 17;
```

8

```
SELECT Product_ID, Size

FROM Product

WHERE Price BETWEEN 200 AND 1000;
```

9

```
UPDATE Customer

SET Address = 'Sylhet'

WHERE Name = 'Jony';
```

10

```
DELETE FROM Customer

WHERE Name = 'Safin';
```